Practical 2: Installing CRD with Docker

Day 2 — Duration: 3 Hours

SDI Training for ICT Professionals — Climate Risk Database

What you will accomplish: By the end of this practical you will have a fully working Climate Risk Database (CRD) running on your own laptop. You will be able to open it in your browser, log in as administrator, and see the GeoNode platform ready for data.

Before you start: Docker Desktop must be running. Check for the whale icon in your system tray (Windows) or menu bar (Mac). If you skipped Practical 1, go back and complete it first.

Step-by-Step Installation

Open your terminal:

- **Windows:** Open **PowerShell** (search in Start menu)
- **Mac:** Open **Terminal** (Applications → Utilities → Terminal)
- **Ubuntu:** Press `Ctrl+Alt+T`

Step 1: Open your terminal and navigate to your home folder:

```
cd ~
```

What this does: Makes sure you are in your home folder, which is the default starting location for your user account.

You should see: No output. Your prompt may show ~ to indicate the home folder.

Step 2: Download the GeoNode project template:

```
git clone https://github.com/GeoNode/geonode-project.git my-crd
```

What this does: Downloads a ready-made project template from GitHub into a new folder called `my-crd`. This template contains all the configuration files Docker needs to build the CRD platform.

You should see:

```
Cloning into 'my-crd'...
remote: Enumerating objects: 1234, done.
remote: Counting objects: 100% (...)
...
Resolving deltas: 100% (...), done.
```

If you see "git: command not found":

- **Windows:** Download and install Git from `https://git-scm.com/download/win`
- **Mac:** Type `xcode-select -install` in the terminal
- **Ubuntu:** Run `sudo apt install -y git`

Step 3: Enter the project folder:

```
cd my-crd
```

What this does: Moves you into the `my-crd` folder you just downloaded. All following commands must be run from inside this folder.

You should see: No output. Your prompt may change to show `my-crd`.

Step 4: Create your configuration file:

```
cp .env.sample .env
```

What this does: Copies the sample configuration file to create your own `.env` file. The `.env` file contains settings like passwords and the website address. The sample file provides sensible defaults.

You should see: No output.

Step 5: Edit the configuration file:

```
nano .env
```

What this does: Opens the configuration file in the nano text editor so you can change important settings.

You should see: The nano editor opens showing many lines of configuration.

Now find and change these three lines (use `Ctrl+W` to search in nano):

Line 1 — Site URL: Find the line that starts with `SITEURL` and change it to:

```
SITEURL=http://localhost/
```

Line 2 — Database password: Find the line that starts with `POSTGRES_PASSWORD` and change it to a strong password:

```
POSTGRES_PASSWORD=my_secure_password_2026
```

Line 3 — Secret key: Find the line that starts with `SECRET_KEY` and change it to any long random string:

```
SECRET_KEY=sdi-training-march-2026-change-this-later
```

Save and exit nano:

- Press `Ctrl+O` then `Enter` to save
- Press `Ctrl+X` to exit

Why change these values?

- **SITEURL:** Tells GeoNode where it is hosted. Since we are running it on our laptop, we use `localhost`.
- **POSTGRES_PASSWORD:** The password for the database that stores all your data. Use something hard to guess.
- **SECRET_KEY:** Used for security (encrypting cookies and sessions). Must be unique for each installation.

Step 6: Build the Docker containers:

```
docker compose build
```

What this does: Reads the project configuration and builds all the software containers. Think of this as Docker assembling all the parts of the CRD — the web server, the database, the map server, and more.

You should see: Many lines of output as Docker downloads and builds components:

```
[+] Building 12/12
 => [django] ...
 => [celery] ...
 => downloading packages ...
 => installing dependencies ...
...
```

This step takes 10–15 minutes depending on your internet speed. This is normal — Docker is downloading several gigabytes of software. Do not close the terminal. If you see progress indicators (downloading, extracting), it is working.

Wait until you see a final summary showing all services built successfully before continuing.

Step 7: Start the CRD platform:

```
docker compose up -d
```

What this does: Starts all the CRD services in the background. The `-d` flag means "detached" — the containers run in the background so you get your terminal back.

You should see:

```
[+] Running 8/8
 - Container my-crd-db4geo-1 Started
 - Container my-crd-rabbitmq-1 Started
 - Container my-crd-geoserver-1 Started
 - Container my-crd-django-1 Started
 - Container my-crd-celery-1 Started
 - Container my-crd-nginx-1 Started
 ...
```

What are all these containers?

- **db4geo:** The PostgreSQL/PostGIS database — stores all your data
- **django:** The GeoNode web application — the website you interact with
- **geoserver:** The map server — serves map images and spatial data
- **nginx:** The web server — handles incoming web requests
- **celery:** The task worker — processes background jobs
- **rabbitmq:** The message broker — coordinates tasks between services

Step 8: Watch the startup logs:

```
docker compose logs -f
```

What this does: Shows the live log output from all containers. The `-f` flag means "follow" — new messages appear as they happen, like watching a live feed.

You should see: Many lines of startup messages from different services. Look for these key indicators:

```
django-1 | ... Applying migrations ...
django-1 | ... Running setup_fixtures ...
geoserver-1 | ... GeoServer started ...
nginx-1 | ... ready to accept connections ...
```

When the messages slow down and you see "ready" messages, the platform is starting up. This typically takes 2–5 minutes after `docker compose up`.

To stop watching logs: Press `Ctrl+C`. This only stops showing logs — the containers keep running.

> **If you see error messages (red text or "ERROR"):** Do not panic. Some initial errors are normal during first startup as services wait for each other. Wait 2–3 minutes and check again. If errors persist, see the Troubleshooting section below.

Step 9: Create an administrator account:

```
docker compose exec django python manage.py createsuperuser
```

What this does: Runs a command inside the Django container to create an admin user account. This account has full control over the CRD platform.

You should see: The system will ask you three questions. Type your answers and press Enter after each:

```
Username: admin
Email address: admin@example.com
Password: ********
Password (again): ********
Superuser created successfully.
```

> **Choose a password you will remember!** You will use this username and password to log in to your CRD. For training purposes, you can use a simple password like `admin123` — but never do this on a real server.

Step 10: Open your CRD in the browser:

Open your web browser (Chrome or Firefox) and go to:

```
http://localhost
```

What this does: Opens the CRD platform running on your own laptop. "localhost" means "this computer".

You should see: The GeoNode homepage with a map, a search bar, and navigation menus. The page may look plain since we have not customised it yet.

Step 11: Log in as administrator:

- Click the **"Sign in"** button (usually top-right corner)
- Enter the username and password you created in Step 9

- Click **"Sign in"**

You should see: You are now logged in. Your username appears in the top-right corner. You should also see an **"Admin"** link in the menu, which only administrators can see.

Useful Docker Commands

Here are commands you will use frequently. You do not need to run them now, but keep this page as a reference.

1. **Check which containers are running:**

```
docker compose ps
```

What this does: Shows a table of all containers and whether they are running or stopped.

2. **Stop the CRD:**

```
docker compose down
```

What this does: Stops all CRD containers. Your data is preserved.

3. **Start the CRD again:**

```
docker compose up -d
```

What this does: Starts all containers again with your existing data.

4. **Restart a single service:**

```
docker compose restart django
```

What this does: Restarts only the web application container without affecting others.

Troubleshooting

Problem: Port 80 is already in use

Symptom: You see an error like "bind: address already in use" or "port 80 is already allocated".

Cause: Another program (like Apache, IIS, or Skype) is using port 80.

Solution:

Step 1: Find what is using port 80:

```
# On Mac/Linux:
sudo lsof -i :80

# On Windows PowerShell:
netstat -ano | findstr :80
```

What this does: Shows which program is using port 80.

Step 2: Stop the conflicting program, or change the CRD port by editing `.env`:

```
nano .env
```

Find the `HTTP_PORT` line and change it:

```
HTTP_PORT=8080
```

Then restart:

```
docker compose down && docker compose up -d
```

Now access CRD at `http://localhost:8080` instead.

Problem: Not enough memory

Symptom: Containers crash or Docker shows memory errors.

Cause: Docker needs at least 4 GB of RAM allocated.

Solution:

- Open Docker Desktop → Settings → Resources
- Set Memory to at least **4 GB** (6 GB recommended)
- Click **Apply & Restart**
- Then run `docker compose up -d` again

Problem: Docker is not running

Symptom: You see "Cannot connect to the Docker daemon" or "docker: command not found".

Solution:

- **Windows/Mac:** Open Docker Desktop from the Start menu or Applications folder. Wait for the whale icon to stop animating.
- **Ubuntu:** Start the Docker service:

```
sudo systemctl start docker
```

What this does: Starts the Docker background service on Ubuntu.

You should see: No output. Then try your `docker compose` command again.

Problem: The website shows an error page

Symptom: The browser shows "502 Bad Gateway" or a GeoNode error page.

Cause: The Django application has not finished starting yet.

Solution: Wait 2–3 more minutes, then refresh the page. Check the logs to see if Django is still starting:

```
docker compose logs django --tail 20
```

What this does: Shows the last 20 lines of logs from the Django container.

You should see: If you see "Booting worker" or "Listening at", Django is running. Refresh your browser.

Checkpoint

Before proceeding to Practical 3, confirm:

☐ **CRD loads:** `http://localhost` shows the GeoNode homepage

☐ **Admin login works:** You can sign in with the account you created in Step 9

- ☐ **Admin panel accessible:** You can visit `http://localhost/admin/` and see the admin interface
- ☐ **All containers running:** `docker compose ps` shows all services as "running"

Well done! You now have a fully functional CRD platform running on your laptop. In the next practical, you will customise it with your institution's name and branding.

End of Practical 2 — Proceed to Practical 3: Customising Your CRD